

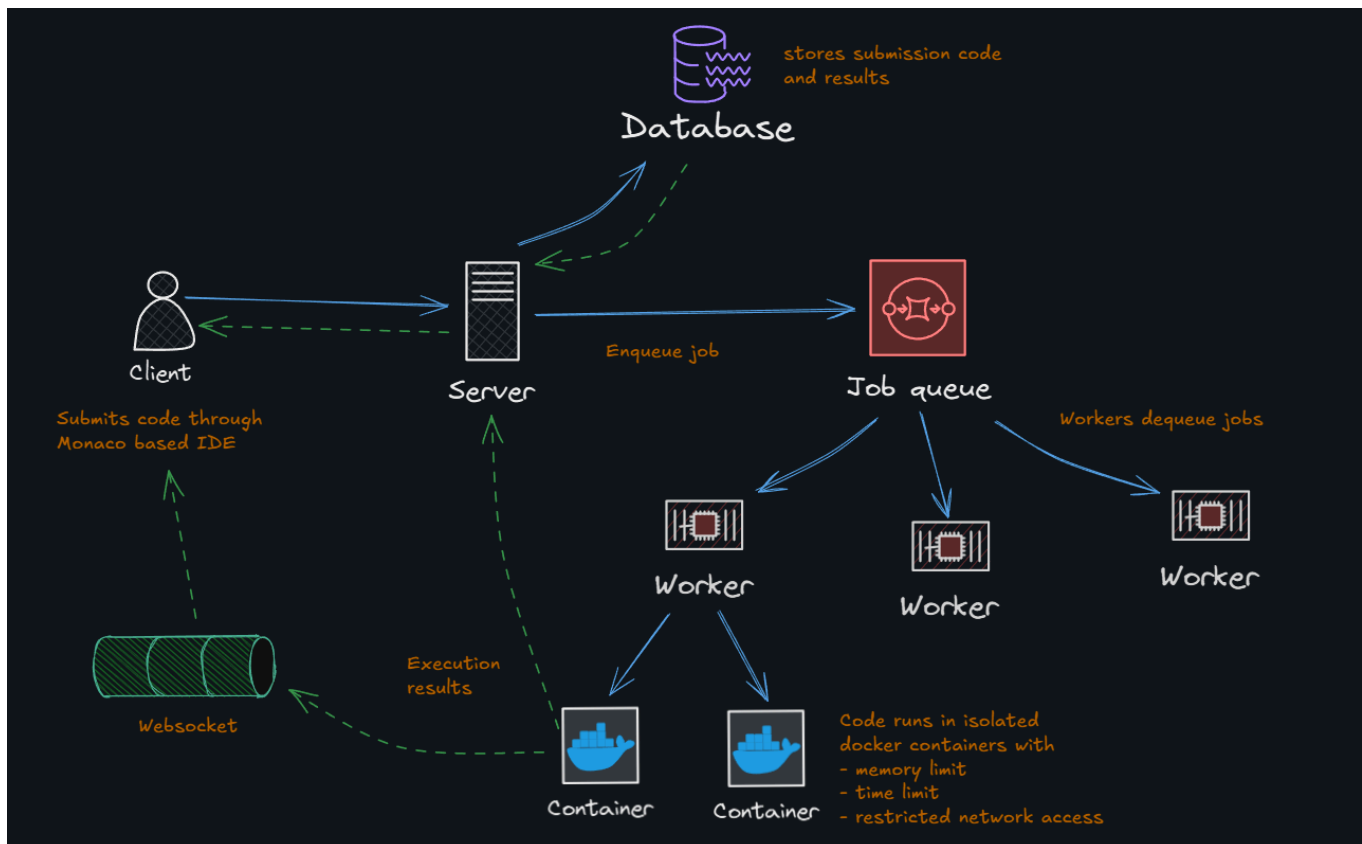
Scalable Code Execution Engine - Design doc

<https://github.com/dibyajyoti-mandal/code-exec-engine>

System Architecture

Main components include :

1. Client - Submits code using a **Monaco** based code editor frontend
2. Gateway server - receives all code jobs and pushes jobs into the job queue
3. Job queue
 - a. Acts as a buffer between the API and Workers.
 - b. Ensures requests are not lost if workers are busy.
4. Worker engine
 - a. Subscribes to the job queue
 - b. Polls for jobs
 - c. Spawns worker nodes
 - d. Worker nodes execute code in isolated containers
5. Websocket - Real time updates for jobs submitted
6. Database - Stores code execution results as well as code files



Tech stack

1. **React** frontend, used **Monaco** library for code editor
2. Worker, gateway and websocket using **Go**
3. **Redis streams** as message queue
4. **Docker** for execution container

Key Design Choices

Why Go?

- Go is widely used in network services.
- Its net/http library and goroutine model allow it to handle tens of thousands of concurrent connections (using goroutines) with very little memory overhead. This is crucial when thousands of users submit code simultaneously during a contest.
- It has a strong standard library reducing dependency on external packages.
- In contrast with Python/Node based applications, Go compiles to machine code. This reduces the CPU cycle cost per request hence giving low latency service.

Redis Streams as message queue

- Operations happen in memory over a persistent TCP connection. Pushing a job (XADD) and popping it (XREADGROUP) takes microseconds.
- You can spin up multiple Worker instances all reading from the job stream (REDIS_STREAM). Redis ensures that each job is delivered to only one worker.
- If a worker crashes while processing a job, it can retry the job once it restarts.

```
func NewRedisPublisher(addr string) *RedisPublisher {
    return &RedisPublisher{
        client: redis.NewClient(&redis.Options{
            Addr: addr,
        }),
        stream: constants.REDIS_STREAM,
    }
}

func (r *RedisPublisher) Publish(job models.Job) error {
    body, _ := json.Marshal(job)

    _, err := r.client.XAdd(context.Background(), &redis.XAddArgs{
        Stream: r.stream,
        Values: map[string]interface{}{
            "job": string(body),
        },
    }).Result()

    if err != nil {
        log.Println("Redis publish error:", err)
    }

    return err
}
```

Docker container as execution runtime

- Isolation - submitted code runs in an isolated linux environment and it cannot see other processes, access the host network, or reach internal APIs. Changes are destroyed on exit, ensuring a clean state for every submission.
- Easy to ensure memory and time limit control to judge MLE or TLE responses.
- Fast boot time and low memory overhead allows high density (running hundreds of containers per server).
- Compared to services like AWS Lambda (based on Firecracker microVM), dockers doesn't have cold-start issues and network restrictions are easier to implement.

```
#!/bin/sh

# Write incoming code to a file
echo "$CODE" > code.cpp

# Compile the C++ program
g++ code.cpp -o code.out

# If compilation fails, exit immediately
if [ $? -ne 0 ]; then
    exit 1
fi

# Run with 1-second time limit
timeout 1 ./code.out
```

```
cmd := exec.CommandContext(
    ctx,
    "docker", "run", "--rm",
    "--memory=256m",
    "--memory-swap=256m",
    "--cpus=0.5",
    "--network=none",
    "-e", "CODE="+code,
    image,
```

Script to run cpp code and docker command that runs the script in corresponding container. We use timeout, --memory and --network parameters/flags to ensure network isolation and time, memory limit.

Concurrency handling

Concurrency is handled through a **Fan-Out / Fan-In pattern** using Goroutines and Channels, with a specific focus on limiting concurrency per worker.

The concurrency pattern is as follows :

1. The Fan-Out - We spawn a fixed number (WORKER_COUNT) of workers as a separate goroutine working in the background.

```
//fire workers

for i := 1; i <= constants.WORKER_COUNT; i++ {
    go workerLoop(i)
}
```

2. We use buffered channels **jobQueue**, **workerLimiter** and **resultQueue** to pass data safely between Goroutines without locks.
 - a. **jobQueue**: Acts as the input buffer. It consumes jobs from the redis stream.
 - b. **workerLimiter**: Limits the number of jobs each worker can execute concurrently.
 - c. **resultQueue**: Acts as the output buffer. Workers write results here, and the **resultBroadcaster** reads them.

```
func workerLoop(workerID int) {
    workerLimiter := make(chan struct{}, 2)
    var mu sync.Mutex
    active := 0

    for job := range jobQueue {
        workerLimiter <- struct{}{}

        mu.Lock()
        active++
        mu.Unlock()

        go func(j models.Job) {
            defer func() {
                <-workerLimiter
                mu.Lock()
                active--
                mu.Unlock()
            }()

            //processing update
            resultQueue <- models.Result{
                JobID:    j.ID,
                ClientID: j.ClientID,
                Status:    "Processing",
            }

            //run job
            res := runJobLogic(workerID, j)
            res.Status = "Completed"
            resultQueue <- res
        }(job)
    }
}
```

Future improvements for production

1. Database integration to store execution results for the users and setting up systems to check against given test cases. The current version solely focuses on isolated and safe execution of the submitted code.
2. Source code and test cases cannot be stored in databases or passed as payload. Usage of **Object storage** like AWS S3 or miniIO as storage for code files and test cases which is suitable for scenarios of competitive programming contests.

3. Migration to **RabbitMQ** or **Kafka** for topic based routing of jobs. We can route code jobs based on language / difficulty of question to workers with different configurations. Redis streams don't offer this feature.
4. We can use **Google gVisor** which works as a secure sandbox for Docker containers. It intercepts application system calls and runs them in a user-space kernel, providing a strong defense-in-depth layer for Docker containers without changing the entire infrastructure.
5. Workers are a static set of Go processes that require manual scaling. Instead we can containerize the Worker service itself and deploy it on **Kubernetes (K8s)** with a **Horizontal Pod Autoscaler (HPA)** for auto scaling based on the number of pending jobs in the message queue.